

Project Final Report:
Mobile Application - TownHappening

Danielle Kemp

Department of Informatics, Indiana University

INFO-C 342 – Mobile Application Development

June 29, 2026

Table of Contents

1) Purpose.....	3
Implemented Functionalities.....	3
2) Major Techniques Used	6
Challenges and Resolutions.....	7
3) New Skills Learned	9
4) Generative AI Usage Disclosure.....	10
5) Test-Run Collection.....	11
Unimplemented Features / Known Bugs	14
References	15

Project Final Report

1) Purpose

TownHappening is a mobile application that is designed to help the residents of small towns discover local events in one centralized location. Many of the current local events that are advertised are done so across several platforms such as Facebook, Instagram, town/government websites, physical flyers, and community bulletin boards. This method makes it difficult for residents to stay informed about activities occurring within their communities. TownHappening addresses this problem by providing a simple and organized platform where users can browse events, view details, save favorites, and share event information with others.

The target audience will include long-time residents, families looking for local activities, young adults interested in community engagement, and newcomers who may not know where to find information about local events.

Implemented Functionalities

Home/browse Screen

The Home Screen is the starting point of the mobile application. It is where the scrollable list of event cards is located using `ListView.builder`. These cards contain:

- Event image
- Event title
- Category badge
- Date and time
- Location
- Attendee count

- Favorite (heart) button

The screen also includes a real-time search bar that filters events by title or location as the user types. Category filter chips allow users to quickly filter events by categories such as Music, Food, Sports, Arts, Community, and Family. The green header at the top displays the current location (Hobart, IN) which the user can tap to open a Change Location dialog, along with the TownHappening app title.

Event Detail Screen

When the user taps on an event card, they are navigated to the Event Detail Screen using `Navigator.push` and `MaterialPageRoute`. The screen uses a `SliverAppBar` with a collapsing green header. The Event Detail Screen displays:

- Event image
- Event title
- Category badge
- Date and time
- Location
- Number of attendees
- Full event description
- Save / Unsave button
- Share button

The share feature was implemented using the `share_plus` package and allows users to share event details including the name, date, location, and description through any supported application on their device.

Favorites Screen

The Favorites Screen displays all favorited events using the same reusable EventCard widget as the Browse Screen, keeping the design consistent. Users can add or remove favorites from any screen and changes update automatically throughout the application. The Favorites Screen includes:

- A green header displaying the title and a count of how many events are favorited
- A scrollable list of all saved event cards
- The ability to remove a favorite by tapping the heart icon directly from this screen
- A friendly empty-state message when no events have been saved yet
- Persistent storage using SharedPreferences so favorites remain saved after the app is closed and reopened
- Event IDs are stored locally on the device rather than full event objects, and are reloaded automatically when the application starts

Additional Features

- Bottom Navigation: A two-tab bottom navigation bar allows users to switch between the Browse and Favorites screens. The Favorites tab displays a live badge showing the current number of favorited events, updating instantly when favorites are added or removed.
- Location Display: The header on the Browse Screen shows the current location and allows the user to tap it to open a Change Location dialog where they can update their town.

- **Share Events:** The share_plus package is integrated into the Event Detail Screen, triggering the native Android share sheet with event details pre-filled including the name, date, location, description, and a Shared via TownHappening tag.

2) Major Techniques Used

- *Flutter Layout System:*

The entire layout of the system was made using the Flutter widgets. We used Row, Column, Expanded, Container, Padding, and ListView.builder. The event cards use a custom Container with rounded corners, shadow, and a Row contained a ClipRRect image on the left and the event details on the right.

- *Navigation:*

The Navigation between all of the screens was created using the Navigator.push() and MaterialPageRoute() which helped pass the event data to the event detail screen.

- *State Management:*

This was created using Provider and ChangeNotifier. The FavoritesProvider managed all of the favorited event IDs which allowed the screens to listen for updates and rebuild automatically whenever a user adds or removes a favorite.

- *Shared Preferences:*

The shared preferences was used to create local persistence. By doing this, the favorite event IDs are able to be stored locally and then reloaded when the application is started.

- *Filter Logic:*

The browse/home screen filters the events using Dart's `where()` method combined with the two conditions of category match and search query match. Both of these use `.toLowerCase()` for case-insensitive comparison.

- *CustomScrollView:*

The Event Detail Screen uses a `CustomScrollView` with the `SliverAppBar` that helps to collapse the header. This green header on the app helps contain the categories and event title so it scrolls with the page content.

Challenges and Resolutions

- 1) The Flutter SDK was stored inside a OneDrive synced folder, which caused Gradle build failures because OneDrive locked build files while syncing. The error message said it was unable to delete certain Kotlin compile cache directories. The resolution was to pause OneDrive syncing temporarily and delete the locked build cache folders manually, then resume development.
- 2) The `share_plus` package required `compileSdk` version 34 or higher, but the project was defaulting to 33. This produced 15 AAR metadata errors at build time. The fix was updating `compileSdk` and `targetSdk` to 36 in `android/app/build.gradle`, and also setting `minSdk` to 21 explicitly instead of using the Flutter default.
- 3) Initially the heart icon on event cards did not update in real time when favorites were toggled. The fix was wrapping the app in a `ChangeNotifierProvider` at the top level in

main.dart, and using `Provider.of<FavoritesProvider>(context)` in each widget that needed to react to favorite changes.

- 4) `SharedPreferences` does not support storing lists of integers directly. The initial implementation tried to store int IDs directly, which caused a type error. The fix was converting IDs to strings using `.map((id) => id.toString()).toList()` before saving, and parsing them back with `int.parse()` when loading.
- 5) Before development began, the Android emulator ran out of available storage space, which prevented the app from installing and running. This caused an initial delay in development. The resolution was switching to a different virtual device with more allocated storage in the Android Virtual Device Manager.
- 6) Early in development, navigating to the Event Detail Screen caused errors because the Event object was not being passed correctly through the route. The fix was passing the Event object directly through the `MaterialPageRoute` constructor using a builder function, ensuring the full event data was available on the detail screen.
- 7) Event cards had inconsistent spacing and overflow issues on smaller screen sizes, particularly with long event titles and location names. This was resolved by wrapping text elements in `Expanded` widgets and using `noWrap` or overflow handling to prevent layout breaking.

- 8) The default Flutter minSdk value was too low for some dependencies. Setting minSdk explicitly to 21 in build.gradle resolved compatibility warnings and ensured the app built cleanly across all targeted Android versions.

3) New Skills Learned

- *Provider Package for Global State Management:*

The course examples demonstrated setState for local state, but TownHappening required state that updates across multiple screens simultaneously. Learning ChangeNotifier and ChangeNotifierProvider to share state globally was a new skill built through reading the Flutter state management documentation and pub.dev.

- *SharedPreferences for Data Persistence:*

While the course introduced SharedPreferences, implementing it for a list of IDs with serialization and deserialization was a new application. Converting between List<int> and List<String> for storage was a practical Dart skill developed during this project.

- *share_plus Package Integration:*

Using share_plus to trigger the native Android share sheet was not covered in class. The implementation required learning how to format a multi-line share string and call Share.share() from within a button's onPressed callback.

- *CustomScrollView and SliverAppBar:*

The collapsing green header on the Event Detail screen uses SliverAppBar inside a CustomScrollView. This widget combination was researched independently through the Flutter Widget Catalog and community tutorials.

- *ClipRRect for Rounded Image Corners:*

Displaying event images with rounded corners on the left side of each card required wrapping `Image.asset` in a `ClipRRect` widget with specific `borderRadius` values. This was discovered through Flutter documentation.

- *Local Image Assets:*

Adding local PNG images to the project required registering each file in `pubspec.yaml` under the `flutter: assets:` section, then referencing them with `Image.asset()`. Managing and organizing a local assets folder was a new workflow.

- *Android Build Configuration:*

Troubleshooting Gradle errors and editing `android/app/build.gradle` to update `compileSdk`, `targetSdk`, and `minSdk` values was an entirely new skill developed out of necessity during the build process.

4) Generative AI Usage Disclosure

The Generative AI used as learning aids were used for debugging assistants and brainstorming support. All code was created by me and untouched by AI. The AI tools used were ChatGPT through IU and Gemini.

- AI was used to help brainstorm the TownHappening feature list.
- When the Flutter build kept failing due to OneDrive file locking and `compileSdk` version conflicts, AI tools helped identify the root cause and suggested specific steps to resolve the issues.
- AI was used to learn how to manage emulator storage in Android Studio, including how to duplicate virtual devices and allocate more storage space when the emulator ran out of room during testing.

- AI was consulted for ideas on where to find appropriate local event photos and to help decide between storing images locally as assets versus using an online service like Firebase Storage. The recommendation was local assets to start with as a beginner then try to use Firebase later if time allowed.
- When Android Studio and Flutter repeatedly crashed and failed to build, AI assisted in identifying which specific build cache folders needed to be deleted to allow the application to run again.

5) Test-Run Collection

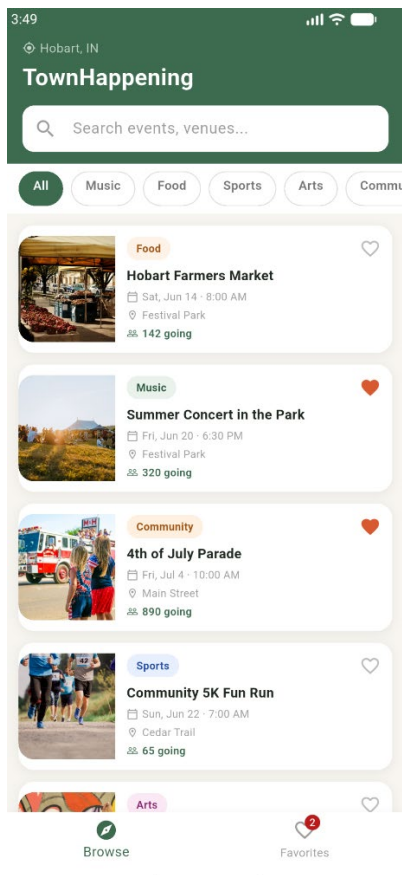


Figure 1- Home Screen

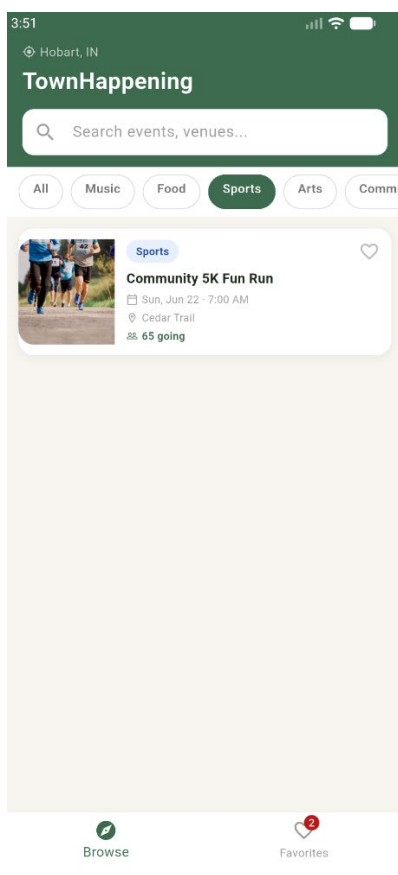


Figure 2- Category Filter

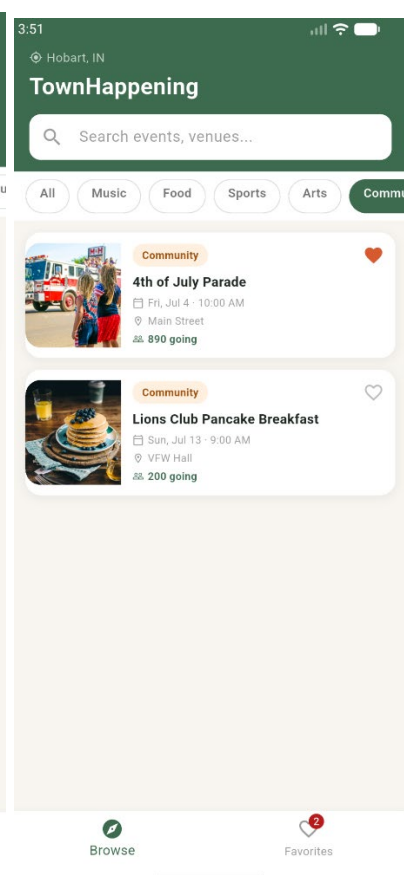


Figure 3- Community Filter

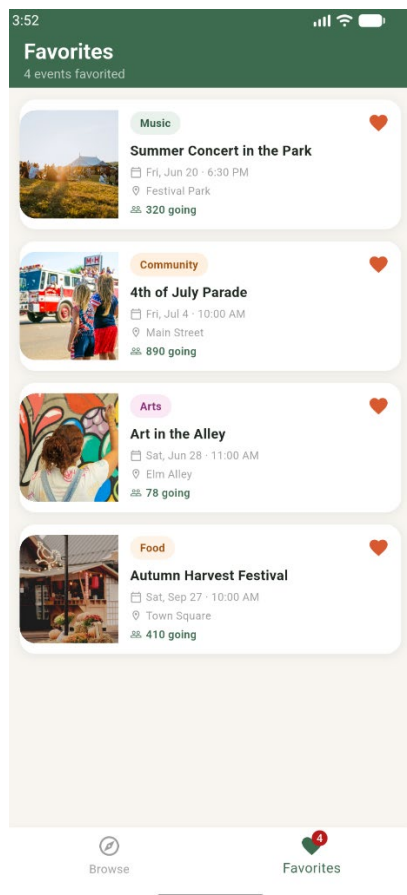


Figure 4- Favorites Screen

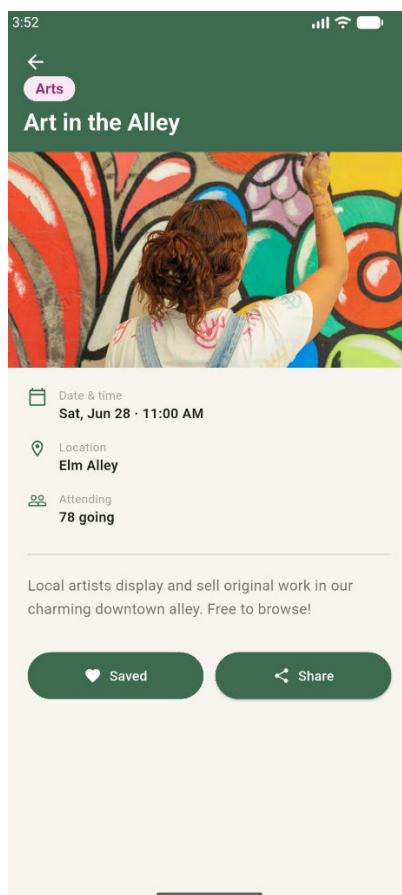


Figure 5- Event Detail Screen

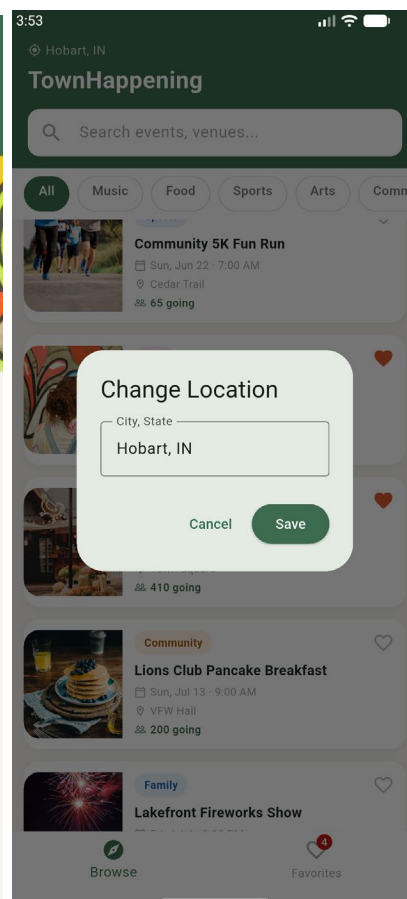


Figure 6 – Change Location

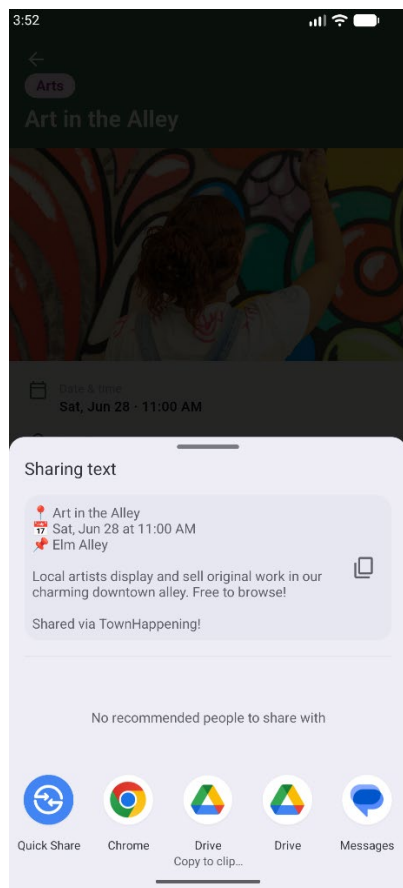


Figure 7- Share Screen

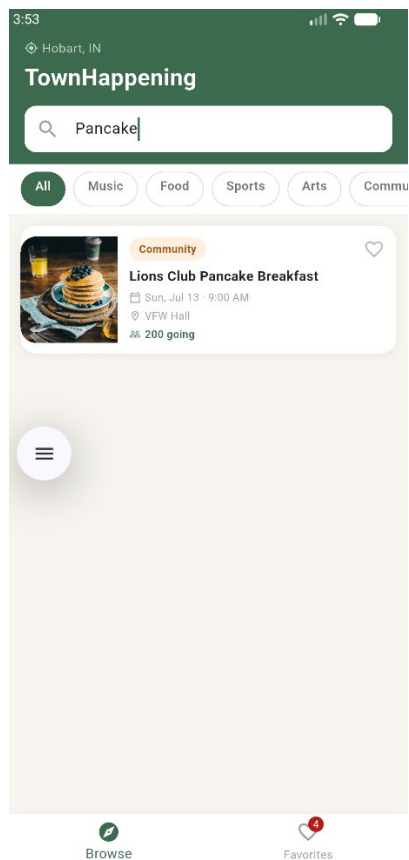


Figure 8- Search Functionality

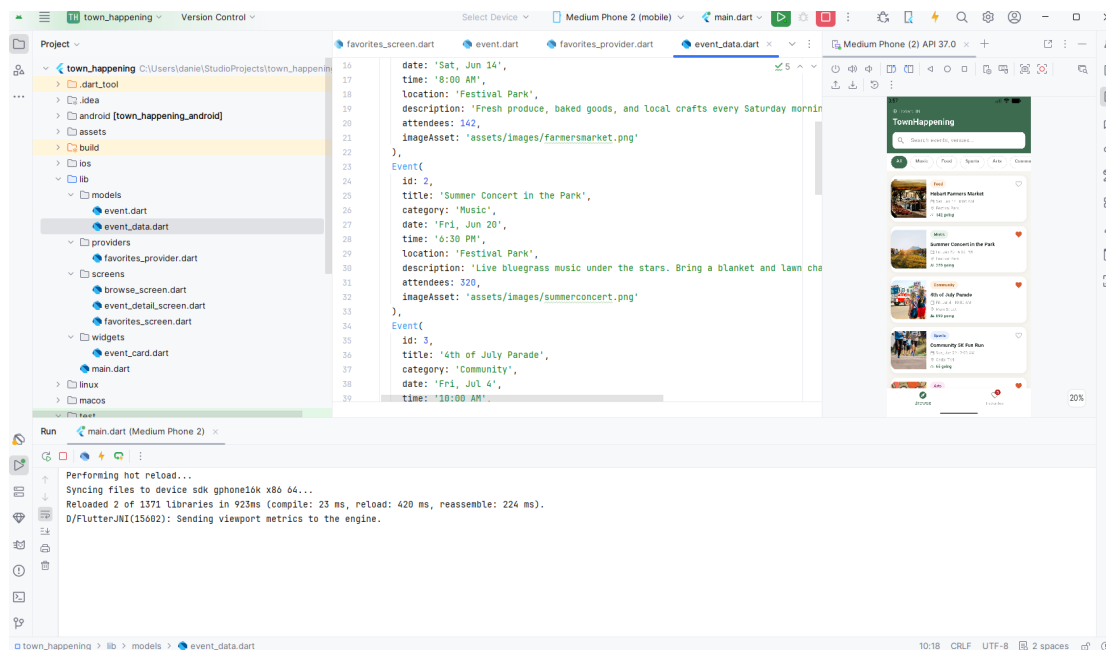


Figure 9 – Working Code Snippet

Unimplemented Features / Known Bugs

- **User Login / Profile Screen:**

The original proposal did not include this feature, but it was considered during development. A login system would allow users to have personalized experiences across devices. This was not implemented as it would require Firebase Authentication, which was beyond the scope of a beginner Flutter project on this timeline.

- **Firebase Firestore Dynamic Data:**

Events are currently hardcoded as Dart objects in `event_data.dart`. A future version would pull live event data from a Firebase Firestore database, allowing real community members or administrators to add events. This was intentionally deferred to keep the project stable and focused on Flutter UI and state management skills.

- **No critical bugs were identified by the end of development. The app is able to run stably on the Android emulator across all three screens with all features functional.**

References

Flutter Documentation

- <https://docs.flutter.dev/>
- <https://docs.flutter.dev/data-and-backend/state-mgmt/simple>

pub.dev Package Pages

- <https://pub.dev/packages/provider>
- https://pub.dev/packages/shared_preferences
- https://pub.dev/packages/share_plus

YouTube Tutorials

- Shared Preferences: <https://www.youtube.com/watch?v=szOllHT1S7Y>
- Flutter Share Files & Text From App:
https://www.youtube.com/watch?v=_7giq9MFD0I
- ListView & Builder: <https://www.youtube.com/watch?v=k1LxTsmAURU>
- Navigator Push: <https://www.youtube.com/watch?v=3F3E-gZ5BF0>
- Flutter Vertical ListView: <https://www.youtube.com/watch?v=uwF0veucf5Q>

AI Tools

- ChatGPT by OpenAI: <https://chatgpt.com>
- Gemini by Google: <https://gemini.google.com>